
Workload-Aware KV-Cache Placement and Scheduling for Low-Latency LLM Inference

Author names to be added

Affiliation and contact details to be added

Sample research manuscript

Abstract

Reusing key–value (KV) state can reduce the work needed to process repeated prompt prefixes, but the location of that state constrains request scheduling. A worker with a useful cache may have a long queue, while a less busy worker may need to fetch the state or recompute it. Changes in arrival rate and prefix popularity can make a previously effective placement expensive. This sample manuscript develops a proposed policy that adapts retained KV-cache placement together with request routing under fixed memory and transfer-bandwidth budgets. The policy compares queueing, exposed transfer time, and remaining computation, then adjusts placement using estimates of future reuse and displacement cost. We specify a system architecture, an online decision procedure, and an evaluation protocol focused on latency during workload transitions. The central question is whether these decisions improve p95 time to first token without degrading decode latency or hiding costs through additional resources. Experiments have not been performed; the results section provides a reporting template rather than empirical findings.

1 Introduction

An inference service must decide both where a request executes and which intermediate state remains available for reuse. These decisions interact. Sending every request to the worker with the longest matching prefix may avoid computation but build a queue at one worker. Sending every request to the least busy worker may spread computation while discarding the benefit of cached work. Transferring a prefix to another worker creates a third possibility, with a cost determined by its size, the available transfer path, and competing traffic.

The trade-off becomes more difficult when demand changes. A document may become popular, a group of conversations may resume together, or an agent workflow may generate a burst of requests sharing a long prefix. Cached state then reflects an earlier workload. A policy that responds only to queue length can miss reuse opportunities, while a policy that moves state too aggressively can consume bandwidth and evict useful blocks.

We study the following working question: *Under fixed memory and transfer-bandwidth budgets, can jointly adapting KV-cache placement and request routing reduce tail latency after traffic or prompt popularity changes?* We use time to first token (TTFT) to measure responsiveness and treat decode

latency as a constraint. The goal is to identify when joint adaptation is worthwhile and when retaining the current placement or recomputing a prefix is preferable.

The proposed study has three intended contributions: (1) a policy that makes routing and retained-state placement decisions using the same cost estimates; (2) a controlled methodology for evaluating workload transitions under equal resource budgets; and (3) an analysis of the regimes in which transferring state improves or harms latency. These are research objectives. Novelty and performance claims remain to be established by implementation, comparison, and measurement.

2 Background and Motivation

2.1 Inference phases and reusable state

During *prefill*, a model processes input tokens and constructs the attention keys and values used by subsequent computation. During *decode*, it generates output tokens while extending that state. For compatible requests with an identical token prefix, a service can retain the corresponding KV blocks and avoid recomputing that prefix. Reuse requires the same model and compatible execution configuration, positions, and attention semantics; matching text alone is insufficient.

GPU high-bandwidth memory (HBM) is close to computation but limited in capacity. Host DRAM provides another location for retained state, with a transfer required before the GPU can consume it. State on another server may traverse a network and a host-to-device path. A hit in a remote cache therefore has a different latency cost from a hit already in local HBM.

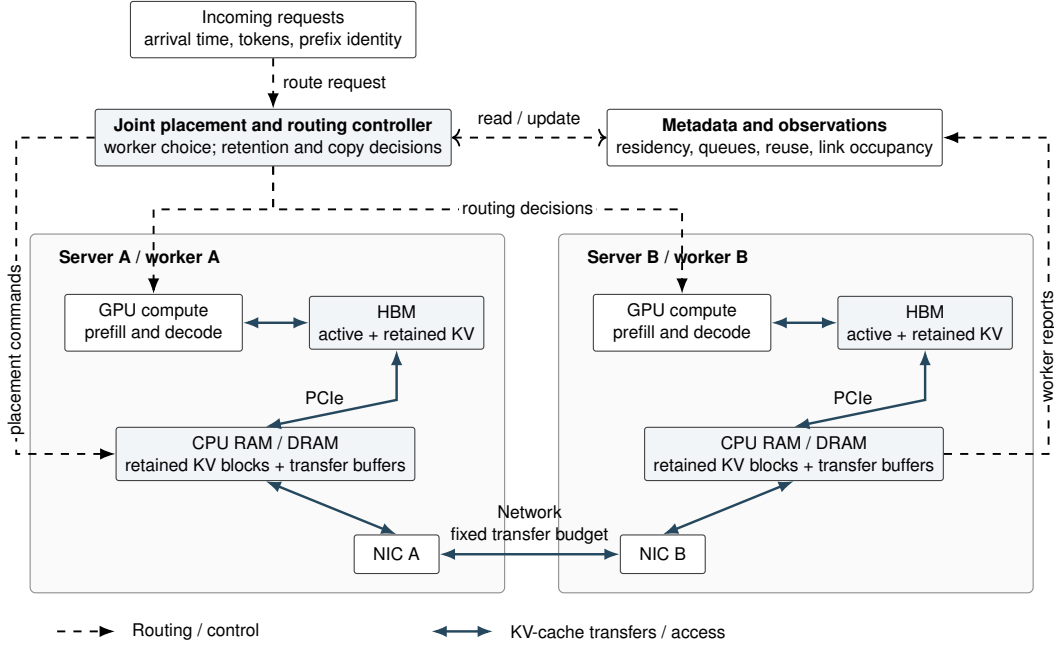


Figure 1: Proposed inference-system architecture, illustrated with two workers. Dashed arrows indicate routing or control; solid arrows indicate KV data movement or access. The controller exchanges metadata rather than carrying the KV payload. Representative control links are shown; both workers receive commands and report state. A remote host-staged transfer follows DRAM–NIC–network–NIC–DRAM–PCIe–HBM. This is a system block diagram, not a proposed chip layout.

2.2 Why the largest cache hit may lose

Suppose worker A holds a prefix but is busy, while worker B is idle. The relevant alternatives are to wait at A, fetch the prefix to B, or recompute at B. The best choice depends on the complete path to the first token. Furthermore, copying the prefix to B may help several later requests, but only if demand persists long enough to repay the transfer and displacement costs. Figure 1 identifies the resources and observations involved in this decision.

3 System Model and Problem Formulation

3.1 Scope and state

We consider one model replicated across a fixed set of workers $\mathcal{W}$. Each worker performs prefill and decode using the same serving engine and local scheduling settings. A worker denotes one serving replica; the initial implementation uses one GPU per replica. The policy controls request assignment and the placement of *retained*, *reusable* prefix state. It does not migrate an active decoding request. Active blocks remain pinned until their users release them.

Request r arrives at time a_r with a known input length n_r and an identifiable token-prefix chain. Its output length is unknown at arrival. Cache object b has size s_b bytes, a configuration identity, a reference count, and a set of verified resident copies. Objects are blocks, but a candidate reuse action must supply a contiguous usable prefix; an isolated block with missing predecessors is not credited as a complete hit.

Table 1: Notation for the proposed policy. Capacities include all cache storage used by a policy.

Symbol	Meaning
$\mathcal{L}$	GPU and host-memory locations across workers
$x_{b\ell}(t)$	Whether a complete copy of object b resides at location ℓ
C_ℓ	Fixed KV storage capacity at location ℓ
B_e	Available KV transfer bandwidth on link e
$\hat{Q}_w$	Estimated delay before request service at worker w
$\hat{P}(r, w, u)$	Estimated remaining prefill time under reuse action u
$\hat{X}(r, w, u)$	Transfer time exposed on the path to the first token
$\hat{N}_b(H)$	Predicted reuse count for object b over horizon H
Δ	Interval between placement-controller updates

3.2 Resource constraints

Every retained replica consumes capacity. Let $A_\ell(t)$ denote pinned active state and $R_\ell(t)$ temporary or reserved transfer storage. The policy must satisfy

$$A_\ell(t) + R_\ell(t) + \sum_{b \in \mathcal{B}_{\text{retained}}(t)} s_b x_{b\ell}(t) \leq C_\ell, \quad \ell \in \mathcal{L}. \quad (1)$$

Temporary copies count before the source is released. Model weights and runtime workspace are held fixed separately. All policies receive the same capacities; a replicated object is never treated as free additional cache.

Let $v_e(t)$ be the aggregate KV data rate on link e , including demand fetches, background copies, and offloads. We require $0 \leq v_e(t) \leq B_e$. A configurable share $\rho_e B_e$ limits background traffic, leaving demand transfers with priority. A network copy that also traverses PCIe is charged against both links. The same limits apply to every baseline.

3.3 Objective and online information

For request r , T_r^{first} is the first-token time minus a_r . The primary outcome is $\text{p95}(\{T_r^{\text{first}} : r \in \mathcal{R}_k\})$ for each predeclared arrival-time window $\mathcal{R}_k$ around a workload change. We also constrain decode latency and report all rejections and incomplete requests. The policy sees past arrivals, current residency, queue summaries, and measured resource utilization. It has no access to future arrivals or realized output lengths. Minimizing a per-request estimate is a heuristic for this objective, not a guarantee of optimal p95 latency.

4 Joint Placement and Scheduling Policy

4.1 Per-request routing

The fast decision path runs when a request arrives. For every candidate worker, it enumerates feasible reuse actions: consume a resident prefix, fetch missing blocks of a compatible prefix, or recompute. A fetch is considered only when destination storage and a transfer path appear available; the selected action then reserves resources atomically. If a source disappears before reservation completes, the controller reevaluates the action rather than treating the cached state as usable.

For worker w and action u , the estimated first-token latency is

$$\hat{T}(r, w, u) = \hat{Q}_w + \hat{X}(r, w, u) + \hat{P}(r, w, u) + \hat{D}_1(r, w, u). \quad (2)$$

Here $\hat{D}_1$ is residual work needed to produce the first token after the modeled prefill. It is zero when that work is already included in $\hat{P}$. The exposed transfer term $\hat{X}$ includes only delay that is not hidden behind queueing or computation. A serial sum of full transfer time and full computation time would overestimate actions with overlap; ignoring transfer queueing would underestimate contention.

The selected pair is

$$(w^*, u^*) = \arg \min_{(w, u) \in \mathcal{F}(r)} \hat{T}(r, w, u), \quad (3)$$

where $\mathcal{F}(r)$ contains actions satisfying capacity, compatibility, and resource constraints. Ties favor fewer transferred bytes and then a deterministic worker order. This initial rule is intentionally simple so that its errors can be measured and its behavior reproduced.

4.2 Estimating costs

Prefill estimates are calibrated from input length, usable prefix length, and batch state. Queue estimates account for pending work and the active decode load rather than using request count alone. Transfer estimates include bytes, path, fixed setup cost, and contention. Calibration and controller tuning use separate traces from the reported evaluation. An implementation must record both the predicted components and their realized values.

4.3 Placement for future requests

Routing changes where newly produced state appears, but it does not by itself choose whether an old prefix should remain, be replicated, or be demoted. At intervals of Δ , a second loop evaluates candidate placement actions using a recent history of reuse. A simple starting estimator is an exponentially weighted moving average:

$$\hat{\lambda}_b^{(k)} = (1 - \alpha)\hat{\lambda}_b^{(k-1)} + \alpha c_b^{(k)} / \Delta, \quad \hat{N}_b(H) = H \hat{\lambda}_b^{(k)}, \quad (4)$$

where $c_b^{(k)}$ counts observed useful reuse opportunities during interval k , and $0 < \alpha \leq 1$. Prefix groups are scored as feasible bundles when blocks must move together. Benefits shared by overlapping prefixes are counted once.

For a candidate action m , define a heuristic net benefit in aggregate request-seconds:

$$\widehat{G}(m) = \sum_{r \in \widehat{\mathcal{R}}_H} \widehat{\delta T}_r(m) - \widehat{C}_{\text{move}}(m) - \widehat{C}_{\text{displacement}}(m). \quad (5)$$

The sum represents predicted delay saved for future requests under the resulting routing choices. Movement cost estimates additional delay imposed on requests by copying and contention; it is not simply a byte count subtracted from a time. Displacement cost estimates future recomputation or fetching caused by evicted objects. Uncertain reuse estimates reduce the confidence in a move, especially immediately after a short burst.

4.4 Bounded adaptation

The placement loop considers retaining an object, copying it to another worker, moving it to host memory, or evicting it. It admits only feasible actions whose estimated benefit exceeds a positive threshold θ . A minimum residence time prevents a recently moved object from immediately moving back, except when correctness or capacity requires release. A byte budget bounds the amount of background movement admitted in each controller interval.

Actions are ranked by estimated benefit per byte on the limiting link. After each admission, the controller updates its reservations and reevaluates overlapping candidates. This avoids simultaneously assigning several moves the same free memory or spare bandwidth. A copy becomes visible to routing only after completion and validation. When uncertainty is high, the controller can keep placement unchanged while continuing to adapt request routing.

Algorithm 1: Proposed online controller

Input: requests, cache directory, worker observations, capacities, and link budgets.

1. **On request arrival:** derive the compatible prefix identity and inspect verified copies.
2. Enumerate worker/reuse pairs, including recomputation; reject infeasible pairs.
3. Estimate first-token latency with Equation (2); select the lowest-cost pair.
4. Reserve resources atomically, fetch if required, and dispatch. Reevaluate on a stale-state conflict.
5. **On request completion:** release active references and update observed costs and reuse history.
6. **Every Δ :** estimate demand over horizon H and generate placement candidates.
7. Score each candidate with Equation (5), including displaced objects and request interference.
8. Admit moves with $\widehat{G}(m) > \theta$ within remaining capacity and background transfer budgets; update reservations after each move.
9. Publish completed, validated copies. Retain the previous state if no candidate is worthwhile.

Decision procedure to implement and evaluate. This algorithm describes a proposed heuristic; no performance guarantee is claimed.

4.5 Correctness and concurrency

A cache identity includes the model revision, adapter identity when applicable, exact token-prefix chain, position and attention configuration, and KV representation. Hashes support lookup but do not replace configuration checks. Active blocks are pinned. Transfers reserve destination space before starting, keep a valid source until completion, and publish residency only after the copy is complete. A failed transfer invalidates the destination and triggers a fresh scheduling decision.

Verification compares reused execution with recomputation under fixed inputs and model settings. Checks cover partial prefixes, repeated prefixes, concurrent eviction, failed transfers, and metadata staleness. Output-token agreement and numerical differences in intermediate or final outputs are reported with explicit tolerances. Successful cache-hit counters alone do not establish correctness.

4.6 Controller overhead

For W workers and at most U candidate actions per worker, the arrival loop evaluates $O(WU)$ scores, excluding prefix lookup. The periodic loop limits itself to the K most relevant prefix groups, bounding its search by the candidate count rather than total historical cache size. The evaluation measures CPU time, metadata traffic, stale-decision retries, and any additional request delay. These costs are included in end-to-end results.

5 Experimental Methodology

5.1 Testbed and reproducibility

The initial experiment uses two to four GPU workers serving an identical model. The final manuscript will report the model revision, precision, GPU and host-memory capacities, link topology, runtime version, cache block size, batching configuration, and controller parameters. A simulator may first explore the policy space, but its results must be labeled separately from hardware measurements. Service-time and transfer models require calibration, and a small single-GPU experiment cannot validate distributed contention by itself.

5.2 Workloads and controlled changes

We consider repeated document prefixes, multi-turn conversations, and mixtures of requests with different input and output lengths. Controlled traces vary arrival intensity and the distribution of prefix identities independently. A popularity-shift trace changes which prefixes receive most requests while preserving the arrival rate. A burst trace changes arrival rate while holding the popularity distribution fixed. A combined trace changes both.

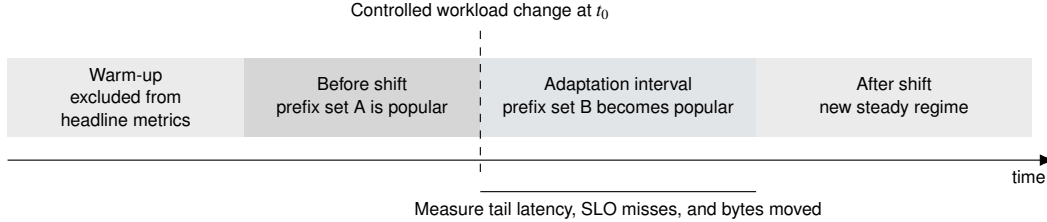


Figure 2: Proposed workload-change protocol. The diagram defines measurement intervals; it contains no measured performance data. Requests are assigned to intervals by arrival time and followed through completion.

Each replay includes a defined warm-up, a pre-change interval, a transition at t_0 , and a post-change interval. Open-loop arrivals preserve offered load even when a policy becomes slow. All policies receive the same trace, prefix contents, random seed, memory budget, and transfer limits. Repeated trials vary the seed and report uncertainty. Parameters are tuned on held-out traces; the test trace is never used to choose a favorable update period or reuse estimator.

5.3 Baselines and ablations

Simple baselines use fixed prefix hashing, least-loaded routing, or longest-prefix-match routing, each with the same cache capacity and eviction mechanism. At least one stronger adaptive scheduler must also be implemented and configured fairly before a claim of improvement is made. A simplified policy reproduction must be distinguished from reproducing an entire serving system.

The primary ablation is factorial: neither controller adapts, routing adapts alone, placement adapts alone, or both adapt. Additional ablations remove transfer-cost accounting, future-reuse estimation, and the minimum-benefit threshold one at a time. Replicated state and staging buffers count toward

memory in every variant. Identical admission rules prevent a method from improving latency merely by rejecting difficult requests.

5.4 Metrics and decision criteria

Report p50, p95, and p99 TTFT; time per output token; end-to-end latency; SLO-compliant completions per second; total transferred bytes; recomputed tokens; and controller overhead. Report failures, rejections, and incomplete requests beside latency statistics. A service-level objective (SLO) is a latency target fixed before comparing policies. Goodput counts successful requests satisfying the chosen first-token and decode targets per unit time.

The main hypothesis is supported only if improvements persist during the transition interval under equal resource budgets, with acceptable decode latency and complete accounting of overhead. Confidence intervals should use independent replays or a method that respects temporal dependence. A higher hit rate without a latency improvement does not establish success.

6 Results and Analysis

Draft status. This section is an explicit reporting template. No experiments have been run for this sample, and no numbers below are benchmark results.

6.1 Overall latency and useful throughput

Table 2 reserves the primary comparison. Each entry will be populated from identical workload replays, with separate tables or panels for steady and changing demand. Report the distribution of first-token latency alongside useful throughput and failure counts. A reduction in p95 TTFT is meaningful only after ruling out differences in admitted load, output lengths, capacity, and run duration.

Table 2: Results template for one fixed-budget workload. A dash means not measured. No values are implied. Decode latency, failures, and uncertainty intervals will accompany the completed table.

Policy	p95 TTFT (ms)	p99 TTFT (ms)	Goodput (req/s)	KV traffic (GB)
Fixed prefix routing	—	—	—	—
Least-loaded routing	—	—	—	—
Longest-prefix routing	—	—	—	—
Adaptive routing baseline	—	—	—	—
Joint placement and routing	—	—	—	—

6.2 Behavior after a workload change

The central figure will plot windowed p95 TTFT before and after t_0 , accompanied by transferred bytes and queue length. The analysis will distinguish a transient spike from a sustained regression. Recovery time is defined in advance as the first point at which the policy satisfies the fixed latency target for a specified number of consecutive windows. Report that duration together with the fraction of requests missing the target; a short recovery can still conceal a severe initial stall.

The intended interpretation is conditional. If a popular prefix persists, creating another copy may repay its cost across later requests. If demand reverses quickly, the same action may waste bandwidth and useful cache space. The experiment must measure both cases and examine whether the controller detects the difference soon enough to matter.

6.3 Separating routing and placement effects

The factorial ablation tests whether the combined policy improves on each component alone. Compare the full controller against adaptive routing with unchanged retention rules, then against adaptive placement under fixed routing. Report both latency and traffic. If most of the benefit comes from routing, the paper should state that directly and narrow its claim about joint adaptation.

6.4 Bandwidth and memory sensitivity

Sweep usable transfer bandwidth, prefix reuse, and KV-memory pressure. A heatmap will show the signed change in p95 TTFT relative to the strongest applicable baseline. The zero-improvement boundary is as useful as the best-performing region: it identifies where the policy should prefer recomputation or stable placement. Increasing host-memory capacity is evaluated separately from changing the policy so that capacity gains are not attributed to scheduling.

6.5 Overhead, errors, and negative results

Report prediction error for queue, transfer, and prefill costs; correlate large errors with poor routing decisions. Include controller CPU time, bytes of metadata, copy cancellations, and migration reversals. No-sharing, short-prompt, and rapid-change workloads are deliberate negative controls. If the joint policy consistently loses on these cases, explain the failure mechanism and the limits of any fallback rule. The sample makes no assumption that one policy wins across all regimes.

7 Related Work

Draft status. Sources and citations are intentionally omitted from this sample. The categories below reserve the comparison structure for the sourced manuscript.

KV memory management. A first group of systems focuses on representing, allocating, sharing, and releasing KV blocks within a serving engine. These mechanisms determine the units that a placement policy can manipulate and the cost of retaining a prefix. The proposed study holds this mechanism fixed while varying the decisions made above it.

Cache-aware request scheduling. A second group routes requests using prefix locality, load, or estimated completion time. Some policies also rebalance requests as conditions change. The relevant comparison is therefore not simply adaptive versus static scheduling. The final paper must establish whether explicit changes to retained-state placement provide additional value beyond a strong adaptive router.

Distributed and tiered cache storage. A third group stores reusable state across GPU memory, host memory, or remote machines and manages the resulting transfers. Capacity, topology, and transfer overlap can dominate the outcome. The proposed distinction to test is the effect of coordinating placement and routing during workload transitions while holding these resources fixed. This distinction remains a hypothesis until the detailed literature comparison is complete.

8 Limitations and Discussion

8.1 Scope of inference

The initial design serves one replicated model and manipulates retained prefixes between requests. Conclusions will not automatically extend to model-parallel replicas, multi-model scheduling, disaggregated prefill and decode, or live migration of active decode state. The illustrated host-staged transfer path is one deployment choice. Other paths may change the cost of moving a prefix and require separate calibration.

8.2 Prediction, fairness, and measurement

Reuse estimation can lag abrupt changes, and queue estimates may become inaccurate when decode load fluctuates. A shared-prefix policy may also favor popular workloads at the expense of rare ones. Report results by workload class and include a waiting-time safeguard in any deployment-oriented implementation. Results from a calibrated simulator remain conditional on its models of overlap, batching, and contention; confidence intervals do not remove structural model error.

The objective also has a practical limitation: optimizing average predicted delay does not directly optimize a tail quantile. Equations (2) and (5) define a tractable heuristic. An observed reduction in p_{95} must be demonstrated experimentally, and the policy should not be described as a solution to a proven global optimum.

8.3 Implications for hardware

The architecture figure exposes the boundary between policy and physical resources. If transfers dominate the critical path, the study may motivate more bandwidth, different staging paths, or dedicated transfer support. If waiting time dominates despite abundant bandwidth, routing and placement may matter more. Such observations could motivate a later hardware study, but this paper does not propose or validate a new chip. Any hardware recommendation must follow from measured bottlenecks.

9 Conclusion

This sample presents a concrete design for studying KV-cache placement and request routing together under fixed resource budgets. It specifies the system state, a two-loop online heuristic, correctness requirements, and an evaluation centered on changing demand. The proposed contribution depends on demonstrating an improvement beyond adaptive routing alone after accounting for transfers, displaced state, and controller overhead. The completed paper should answer both when joint adaptation reduces tail latency and when a simpler policy is preferable.